計算機科学実験及演習 3 ハードウェア

ユーザーズマニュアル

SIMPLE/B 拡張プロセッサシステム

学籍番号　（学籍番号を記入）

氏名　（氏名を記入）

提出日　2026 年 6 月 5 日

京都大学 工学部情報学科 計算機科学コース

目次

1 **システム概要** 2
- 1.1 対象ハードウェア 2

2 **操作方法** 2
- 2.1 起動と実行 2
- 2.2 入力デバイス 2
- 2.3 出力デバイス 2
- 2.4 7 セグメント表示モード 2

3 **命令セットリファレンス** 3
- 3.1 命令形式 3
- 3.2 レジスタ 3
- 3.3 演算命令 4
- 3.4 メモリアクセス命令 4
- 3.5 即値・分岐命令 4
- 3.6 拡張命令 4

4 **I/O ポートマップ** 5

5 **タイマの使い方** 6
- 5.1 レジスタ 6
- 5.2 プリスケーラ 6
- 5.3 使用例: 時間計測 6
- 5.4 使用例: 一定時間待ち 7

6 **アセンブラの使い方** 7
- 6.1 概要 7
- 6.2 実行方法 7
- 6.3 アセンブリ言語の文法 8
- 6.4 命令記法一覧 8

7 **プログラムの開発手順** 8
- 7.1 シミュレーション 8
- 7.2 FPGA 実装 9

8 **サンプルプログラム** 9
- 8.1 Hello World（即値加算と表示）. 9
- 8.2 DIP スイッチ読取り 9
- 8.3 LED 制御 9

8.4 バブルソート 10

9 **トラブルシューティング** 10

1　システム概要

本プロセッサシステムは SIMPLE アーキテクチャ（ver 4.0）に基づく 16 ビットプロセッサであり、以下の拡張を施している。

- 4 サイクル実行方式（基本 5 サイクルから 25% 高速化）
- ISA 拡張: BAL（サブルーチン呼出）、BR（間接ジャンプ）、ADDI（即値加算）
- ハードウェアタイマ（プリスケーラ付き 16 ビットカウンタ）
- I/O アドレス空間（最大 16 ポート、6 ポート実装済み）
- 8 桁 7 セグメント LED 表示（4 モード切替）
- LED 出力（8 ビット）

1.1　対象ハードウェア

表 1　ハードウェア仕様

項目	内容
FPGA ボード	PowerMedusa MU500-RX/RK
FPGA	Altera Cyclone IV EP4CE30F23I7N
クロック	20 MHz 発振器
主記憶	Block RAM 4096 語 × 16 ビット

2　操作方法

2.1　起動と実行

1. FPGA ボードの電源を投入する
2. Quartus Prime から FPGA にプログラムを書き込む（`.sof` ファイル）
3. リセットボタン（`rst_n`、負論理）を押して初期化する
4. 実行ボタン（`exec`、正論理）を押すとプログラムが実行開始する
5. HLT 命令でプロセッサが停止する。再実行するにはリセット後に再度 exec を押す

2.2　入力デバイス

2.3　出力デバイス

2.4　7 セグメント表示モード

ロータリースイッチの下位 2 ビットで表示内容を切り替える。

表 2　入力デバイス一覧

デバイス	接続	備考
DIP スイッチ A (SW26)	`sw[7:0]`	負論理 (ON=0)、I/O ポート 0 で読出し
DIP スイッチ B (SW27)	`sw[15:8]`	負論理 (ON=0)、I/O ポート 1 で読出し
プッシュスイッチ	`pushsw[3:0]`	負論理 (押下=0)、I/O ポート 4 で読出し
ロータリースイッチ	`rotsw[3:0]`	I/O ポート 5 で読出し、表示モード切替兼用

表 3　出力デバイス一覧

デバイス	接続	備考
7 セグメント LED	`seg[7:0]`, `sel[7:0]`	8 桁 HEX 表示、ダイナミック点灯
LED	`led[7:0]`	I/O ポート 4 で書込み

表 4　7 セグ表示モード

ロータリー値	モード	表示内容
0	OUT 値表示	OUT ポート 0/1 の値を 8 桁 HEX 表示（デフォルト）
1	PC 表示	プログラムカウンタの値（デバッグ用）
2	レジスタ表示	r0~r3 の下位 8 ビットを各 2 桁で表示
3	タイマ表示	タイマ COUNT 値

3　命令セットリファレンス

3.1　命令形式

全命令は 16 ビット固定長であり、4 種類の形式を持つ。

表 5　命令形式

形式	ビット配置	用途
(a)	`11|Rs(3)|Rd(3)|op3(4)|d(4)`	演算, シフト, IN/OUT, HLT
(b)	`op1(2)|Ra(3)|Rb(3)|d(8)`	LD (op1=00), ST (op1=01)
(c)	`10|op2(3)|Rb(3)|d(8)`	LI, BAL, BR, ADDI, B
(d)	`10|111|cond(3)|d(8)`	Bcc (条件分岐)

3.2　レジスタ

8 本の汎用レジスタ `r0`~`r7`（各 16 ビット）を持つ。慣例として `r7` を BAL 命令のリンクレジスタとして使用する。

3.3 演算命令

表 6: 演算命令一覧

ニーモニック	op3	動作	フラグ
ADD Rd, Rs	0000	r[Rd] = r[Rd] + r[Rs]	S,Z,C,V
SUB Rd, Rs	0001	r[Rd] = r[Rd] - r[Rs]	S,Z,C,V
AND Rd, Rs	0010	r[Rd] = r[Rd] & r[Rs]	S,Z
OR Rd, Rs	0011	r[Rd] = r[Rd] | r[Rs]	S,Z
XOR Rd, Rs	0100	r[Rd] = r[Rd] ^ r[Rs]	S,Z
CMP Rd, Rs	0101	r[Rd] - r[Rs]（フラグのみ）	S,Z,C,V
MOV Rd, Rs	0110	r[Rd] = r[Rs]	S,Z
SLL Rd, d	1000	左論理シフト	S,Z,C
SLR Rd, d	1001	左循環シフト	S,Z
SRL Rd, d	1010	右論理シフト	S,Z,C
SRA Rd, d	1011	右算術シフト	S,Z,C
IN Rd, port	1100	r[Rd] = input[port]	—
OUT Rs, port	1101	output[port] = r[Rs]	—
HLT	1111	実行停止	—

3.4 メモリアクセス命令

表 7　ロード/ストア命令

ニーモニック	動作
LD Ra, d(Rb)	r[Ra] = mem[r[Rb] + sign_ext(d)]
ST Ra, d(Rb)	mem[r[Rb] + sign_ext(d)] = r[Ra]

変位 d は 8 ビット符号付き（−128~+127）。

3.5 即値・分岐命令

3.5.1 条件分岐の条件

3.6 拡張命令

3.6.1 BAL/BR の使い方

BAL はサブルーチン呼び出しに、BR はサブルーチンからの復帰に使用する。

```
; メインプログラム
```

表 8　即値・分岐命令

ニーモニック	op2	動作
`LI Rb, d`	000	`r[Rb] = sign_ext(d)`
`B d`	100	`PC = PC + 1 + sign_ext(d)`
`BE/BLT/BLE/BNE d`	111	条件成立時 `PC = PC + 1 + sign_ext(d)`

表 9　条件分岐コード

ニーモニック	cond	条件
`BE`	000	$Z = 1$（等しい）
`BLT`	001	$S \oplus V = 1$（符号付き小なり）
`BLE`	010	$Z \vee (S \oplus V) = 1$（符号付き以下）
`BNE`	011	$Z = 0$（等しくない）

表 10　ISA 拡張命令

ニーモニック	op2	動作	フラグ
`BAL Rb, d`	001	`r[Rb] = PC; PC = PC + 1 + sign_ext(d)`	—
`BR Rb`	010	`PC = r[Rb]`	—
`ADDI Rb, d`	011	`r[Rb] = r[Rb] + sign_ext(d)`	S,Z,C,V

```
        BAL  r7, sub        ; sub を呼び出し（r7 = 戻りアドレス）
        OUT  r0              ; sub から戻った後に実行される
        HLT

; サブルーチン
sub:    LI   r0, 42
        BR   r7              ; r7 の値（=呼出元の次の命令）に戻る
```

Listing 1　サブルーチン呼出の例

4　I/O ポートマップ

IN/OUT 命令の第 2 オペランドでポート番号を指定する。省略時はポート 0 が使用される（後方互換）。

表 11　I/O ポートマップ

ポート	IN (読み出し)	OUT (書き込み)	アセンブラ記法
0	DIP スイッチ A (8bit)	7 セグ出力 下位 16bit	`IN r0, 0`
1	DIP スイッチ B (8bit)	7 セグ出力 上位 16bit	`IN r0, 1`
2	タイマ COUNT 値 (16bit)	タイマ CTRL レジスタ	`OUT r0, 2`
3	タイマ overflow (bit0)	タイマ LIMIT 値	`OUT r0, 3`
4	プッシュスイッチ (4bit)	LED 出力 (8bit)	`OUT r0, 4`
5	ロータリースイッチ (4bit)	(予約)	`IN r0, 5`

表 12　タイマレジスタ

レジスタ	ポート	R/W	説明
COUNT	IN 2	R	現在のカウント値（16 ビット）
CTRL	OUT 2	W	bit0: 動作 (1)/停止 (0), bit1: リセット, bit[15:8]: プリスケーラ
LIMIT	OUT 3	W	カウント上限値
overflow	IN 3	R	bit0: 上限到達フラグ (読出しで自動クリア)

5　タイマの使い方

5.1　レジスタ

5.2　プリスケーラ

カウント周波数 $= 20\,\mathrm{MHz} \div (\mathrm{PRESCALER} + 1)$。

- PRESCALER = 0: 20 MHz（50 ns 間隔）
- PRESCALER = 19: 1 MHz（1 μs 間隔）
- PRESCALER = 199: 100 kHz（10 μs 間隔）

5.3　使用例: 時間計測

```
; タイマ初期化
        LI   r1, 0xFF        ; LIMIT = 0x00FF（上位=0）
        OUT  r1, 3            ; LIMIT レジスタに設定

; カウント開始（プリスケーラ=0, 20MHz）
        LI   r1, 1            ; CTRL: bit0=1（動作開始）
        OUT  r1, 2

```

```
; ...（計測対象の処理）...

; カウント値読出し
        IN   r0, 2            ; r0 = COUNT値
        OUT  r0               ; 7セグに表示

; タイマ停止
        LI   r1, 0
        OUT  r1, 2            ; CTRL = 0（停止）
```

Listing 2　タイマによる実行時間計測

5.4　使用例: 一定時間待ち

```
; LIMIT=9999, プリスケーラ=199 で 1秒タイマ
; (100kHz / 10000 = 10Hz → 100ms, LIMIT=9999でCOUNT 0~9999)
        LI   r1, 0x0F         ; LIMIT下位（仮）
        ;（...上位ビットの設定はLI+SLL+ORで構築...）
        OUT  r1, 3             ; LIMIT設定

; CTRL = プリスケーラ199(=0xC7), EN=1 → 0xC701
        ;（...16ビット定数の構築...）
        OUT  r1, 2             ; タイマ動作開始

; overflow待ちループ
wait:   IN   r0, 3             ; overflow読出し
        LI   r1, 1
        AND  r0, r1            ; bit0を取得
        BE   wait              ; overflow=0 なら待ち継続
        ; overflow=1 で脱出（読出し時に自動クリア済み）
```

Listing 3　タイマによる一定時間待ち

6　アセンブラの使い方

6.1　概要

simple_asm.py は全命令に対応した Python アセンブラである。ラベル、コメント、PC 相対分岐のアドレス自動計算をサポートする。

6.2　実行方法

```
# 基本的な使い方
python3 simple_asm.py input.asm -o output.hex

# リスティング付きで実行
python3 simple_asm.py input.asm -o output.hex -l
```

```

# 出力ファイル名を省略すると input.hex が生成される
python3 simple_asm.py input.asm
```

Listing 4　アセンブラの実行

6.3　アセンブリ言語の文法

```
; 行コメント（セミコロン以降が無視される）
label:  MNEMONIC  operand1, operand2   ; インラインコメント

; ラベル定義: 行頭の識別子 + コロン
; 数値リテラル: 10（10進）, 0x0A（16進）, 0b1010（2進）
; 負の即値: -1, -128
; レジスタ: r0 ~ r7（大文字/小文字可）
```

Listing 5　アセンブリ言語の文法

6.4　命令記法一覧

表 13　アセンブラ記法

記法	例
ADD Rd, Rs	ADD r0, r1
LD Ra, d(Rb)	LD r0, 5(r1) / LD r0, -3(r2)
ST Ra, d(Rb)	ST r0, 0(r1)
LI Rb, imm	LI r0, 0x80 / LI r1, -1
B label	B loop
Bcc label	BLT inner / BNE done
BAL Rb, label	BAL r7, func
BR Rb	BR r7
ADDI Rb, imm	ADDI r0, 1 / ADDI r3, -1
IN Rd [, port]	IN r0 / IN r0, 2
OUT Rs [, port]	OUT r0 / OUT r1, 4
HLT	HLT

7　プログラムの開発手順

7.1　シミュレーション

```
cd simple/

# アセンブル
python3 simple_asm.py myprog.asm -o myprog.hex -l

```

```
# コンパイル（INIT_FILE を指定）
iverilog -DINIT_FILE=\"myprog.hex\" -o sim.out \
    simple_tb.v simple_cpu.v alu.v shifter.v regfile.v ram_sim.v

# 実行
vvp sim.out
```

Listing 6　iverilog によるシミュレーション

7.2　FPGA 実装

1. アセンブラで hex ファイルを生成
2. hex ファイルから MIF ファイルを作成し、`run.mif` として保存
3. Quartus Prime でプロジェクトを開き、全 Verilog モジュールを追加: `simple_top.v`, `simple_cpu.v`, `alu.v`, `shifter.v`, `regfile.v`, `ram.v`, `seg7dec.v`, `timer.v`
4. ピンアサインを設定（`.qsf` ファイル）
5. SDC ファイルでクロック制約を設定
6. コンパイル → Programmer で FPGA に書き込み

8　サンプルプログラム

8.1　Hello World（即値加算と表示）

```
; hello.asm - 3 + 5 = 8 を7セグに表示
        LI   r0, 3          ; r0 = 3
        LI   r1, 5          ; r1 = 5
        ADD  r0, r1         ; r0 = r0 + r1 = 8
        OUT  r0              ; 7セグに "0008" 表示
        HLT                  ; 停止
```

Listing 7　最初のプログラム

8.2　DIP スイッチ読取り

```
; dipsw.asm - DIPスイッチの値を読み取って表示
loop:   IN   r0, 0          ; DIPスイッチAを読取り
        IN   r1, 1          ; DIPスイッチBを読取り
        SLL  r1, 8          ; 上位8ビットにシフト
        OR   r0, r1         ; r0 = B:A (16ビット)
        OUT  r0              ; 7セグに表示
        B    loop            ; 無限ループ
```

Listing 8　DIP スイッチの値を表示

8.3　LED 制御

```
; ledcount.asm - LEDを0~255でカウントアップ
        LI   r0, 0          ; カウンタ
loop:   OUT  r0, 4          ; LEDに出力（ポート4）
        OUT  r0              ; 7セグにも表示
        ; 遅延（ソフトウェアループ）
        LI   r1, 0
delay:  ADDI r1, 1
        BNE  delay           ; r1 がオーバーフロー(0)するまで
        ADDI r0, 1           ; カウンタ++
        B    loop
```

Listing 9　LED をカウントアップ

8.4　バブルソート

`sort.asm` を参照。16 要素の配列をバブルソートし、結果を 7 セグに表示するデモプログラム。

9　トラブルシューティング

表 14　よくある問題と対処法

症状	対処法
exec を押しても動かない	リセットボタンを押してから再度 exec を押す
7 セグが何も表示しない	ロータリースイッチがモード 0 になっているか確認。OUT 命令がプログラムに含まれているか確認
シミュレーションで期待と違う出力	リスティング (`-l`) でアドレスを確認。分岐先ラベルの綴りを確認
アセンブルエラー「即値が範囲外」	8 ビット即値は −128~+127（符号付き）または 0~255（符号なし）。16 ビット定数は LI+SLL+OR で構築する
DIP スイッチの値が反転する	負論理。ハードウェア側で反転済み（`~sw`）なので、ソフトウェアではそのまま使用可能